# Código

Monday, April 20, 2026 5:05 PM

## 1. Seções

- Um programa é dividido em seções
- Seções comuns são:
  - .data - Local para variáveis inicializadas
  - .bss - Local para variáveis não inicializadas
  - .text - Local para código (instruções)

## Rótulos e comentários

- Rótulos marcam posições no código
  - A diretiva .global aponta para um rótulo e define o entrypoint do programa
- Comentário são bytes ignorados pelo assembler

## 2. Programa

NOP

Instrui a CPU a não fazer **absolutamente nada** durante um ciclo de clock, exceto avançar o ponteiro de instrução (*Instruction Pointer*) para a próxima linha.

- Se o seu código de máquina termina em um endereço "quebrado", o Assembler insere vários **NOPs** para preencher o espaço até o próximo endereço alinhado. Isso garante que a próxima função importante comece exatamente onde a CPU consegue ler mais rápido.
- **O Problema:** Às vezes, a Instrução B precisa de um resultado que a Instrução A ainda não terminou de calcular (isso é um *Data Hazard*). **A Solução:** O compilador (ou o hardware) pode inserir um **NOP** entre elas para "dar um respiro" e esperar que o dado fique pronto antes que a Instrução B tente usá-lo.
- Em sistemas muito simples (como microcontroladores), onde você sabe exatamente quantos microssegundos cada ciclo de clock leva, você pode usar uma sequência de NOPs para criar um atraso preciso. **Exemplo:** Se você precisa esperar exatos 3 ciclos para um componente de hardware reagir, você coloca três instruções `NOP`.
- Em sistemas multiprocessados ou ao interagir com hardware externo (como uma placa de vídeo ou um sensor), o NOP pode servir como um espaço reservado enquanto o processador espera um sinal de pronto ou sincroniza sua velocidade com a de um periférico mais lento.

## 3. Inline Code

- Escrita de assembly direto no meio do código C

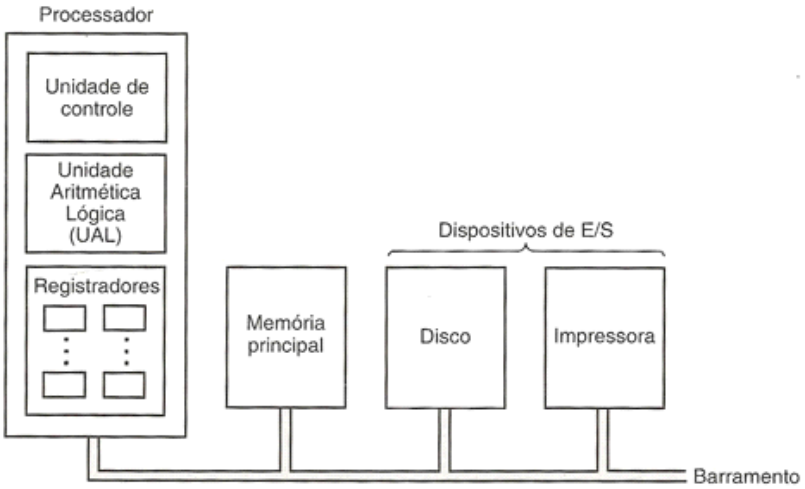
## 4. Montador GAS

- Utilizado no GCC
- Sintaxe padrão é AT&T
- Apesar da sintaxe ser AT&T, o GAS possui um comando para trocar a sintaxe: .intex_sintax e .att
- Também é possível não utilizar prefixos com noprefix

# Registradores

Monday, April 20, 2026 5:20 PM

Registradores são pequenas áreas de armazenamento de memória embutidas no processador

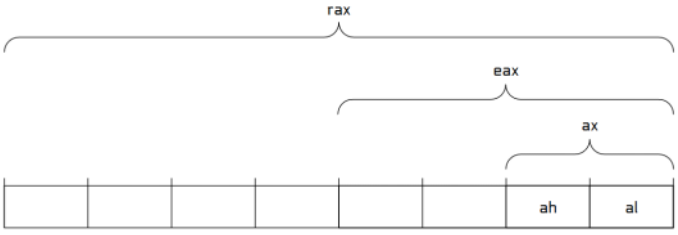



O processador Intel possui registradores:

- De propósito geral (RAX, RBX, ..., R15)
- De Propósito especial (RIP, RFLAGS)
- De segmentos (CS, SS, DS, ES, FS, GS)
- De extensão

a **compatibilidade retroativa** através do acesso parcial a registradores.

- Registradores podem ser acessados de maneira parcial
  - Forma da intel de manter o legado enquanto estende o acesso




- **RAX (64 bits):** É o registrador completo. O "R" vem de *Register* (ou *Record*), padrão para a era de 64 bits.
- **EAX (32 bits):** São os 32 bits inferiores do RAX. O "E" vem de *Extended*. Se você rodar um programa de 32 bits hoje, ele usará o EAX sem saber que o restante do RAX existe.
- **AX (16 bits):** São os 16 bits inferiores do EAX. Era o registrador principal na era do processador 8086.
- **AH e AL (8 bits cada):** O AX é dividido ao meio.
- **AH** (*High*): Os 8 bits superiores do AX.
- **AL** (*Low*): Os 8 bits inferiores do AX.

# Memória

Monday, April 20, 2026 5:31 PM

Apenas um **vetor linear e gigantesco de bytes**, numerados de 0 até o limite da sua RAM.

## 1. Trecho consecutivo de bytes

Para a CPU, a memória não tem estrutura de "pastas" ou "arquivos". Ela enxerga apenas uma sequência infinita de endereços (ex: 0x00, 0x01, 0x02...).

- Se você tem um **inteiro** de 4 bytes, ele ocupará 4 endereços consecutivos.
- Se você tem um **array**, todos os elementos estarão colados uns nos outros.
- É por isso que, em C, se você incrementar um ponteiro, você pula para o próximo byte consecutivo.

## 2. Endereços Computados

Diferente de um registrador (que você chama pelo nome, como `EAX`), a memória precisa ter seu endereço **calculado**. A CPU precisa de uma fórmula para saber em qual prateleira do "vetor de bytes" ela deve buscar o dado.

## 3. Modos de Endereçamento

O modo de endereçamento é a "fórmula" usada para calcular esse endereço final.

# Imediato

Monday, April 20, 2026 5:33 PM

O tipo mais simples de operando em Assembly: o **Imediato**.

Como o nome sugere, o valor está "imediatamente" disponível dentro da própria instrução. A CPU não precisa consultar a RAM ou um registrador para descobrir qual é o número; ele já faz parte do código de máquina.

## 1. O que são na prática?

São as constantes "hardcoded" do seu programa.

- **Em C:** Quando você escreve x = 10;, o número 10 será um operando imediato no Assembly.
- **Notação:** O cifrão ($) indica a sintaxe **AT&T** (do GAS). Na sintaxe Intel (NASM), usaria apenas o número puro (ex: 0x400).

## 2. Codificação (O tamanho importa)

Este é o ponto mais interessante tecnicamente. O Assembler tenta economizar o máximo de espaço possível no arquivo executável:

- **1 byte:** Se você quer somar 5 ($5), o Assembler gasta apenas 1 byte para guardar esse valor.
- **8 bytes:** Se você quer carregar um endereço de memória de 64 bits completo, o valor imediato ocupará 8 bytes dentro da instrução.

  **Exemplo:** A instrução mov rax, $0x1234567890ABCDEF será uma instrução muito "longa" fisicamente na memória, porque carrega 8 bytes de dado imediato nela.

# Instruções de Movimentação

Monday, April 20, 2026 6:09 PM

## 1. A Instrução MOV (Movimentação)

Apesar do nome *move*, ela funciona como um **copy**. Ela copia o dado da origem para o destino, mantendo o valor original intacto.

- **Regra de Ouro:** Você pode mover de *Registrador para Registrador*, de *Imediato para Registrador* ou de *Memória para Registrador*.
- **A Única Proibição:** No x86, você **não pode** mover dados diretamente de um endereço de memória para outro endereço de memória em uma única instrução mov. Você deve primeiro mover para um registrador e depois para o destino.

## 2. Operações Aritméticas e Lógicas

É aqui que o processamento acontece de fato.

- **ADD / SUB:** Soma ou subtrai o valor da origem no destino. O resultado fica guardado no destino.
- **MUL / DIV:** Multiplicação e divisão. São instruções mais complexas que geralmente usam registradores fixos (como o eax e edx) para guardar o resultado e o resto.
- **SHL / SHR:** *Shift Left* (Deslocamento para a esquerda) e *Shift Right*. São formas ultra rápidas de multiplicar ou dividir por potências de 2 (2, 4, 8, 16...).

## 3. Manipulação de Memória e XCHG

Diferente do mov, o **XCHG** (*Exchange*) é uma operação atômica que troca os valores entre dois lugares simultaneamente.

- **Uso em Multithreading:** O XCHG é muito importante em sistemas operacionais para criar "semáforos" ou "locks". Ele garante que dois processos não tentem usar o mesmo recurso ao mesmo tempo, pois a troca acontece de uma só vez, sem interrupções.

## 4. A Instrução LEA (Load Effective Address)

Esta é, talvez, a instrução mais "truqueira" e útil do x86. **LEA** significa "Carregar Endereço Efetivo".

**A diferença para o MOV:**

- MOV EAX, [EBX + 8] -> Vai até a memória no endereço EBX+8, pega o que está lá dentro e coloca em EAX.
- LEA EAX, [EBX + 8] -> Não vai na memória. Ela apenas faz o cálculo matemático EBX + 8 e coloca o **resultado do endereço** no EAX.

**Curiosidade Pro:** Programadores de Assembly (e compiladores) usam o LEA para fazer contas matemáticas rápidas, mesmo que não envolvam memória. Como o LEA consegue somar dois registradores e multiplicar um deles por uma escala (1, 2, 4 ou 8) em um único ciclo, ele é mais rápido que usar ADD e MUL separadamente.

Exemplo de "truque" com LEA: lea eax, [ebx + ebx*4] é uma forma ultra rápida de fazer eax = ebx * 5.

# Movimentação de Dados

Monday, April 20, 2026 6:14 PM

A CPU precisa saber exatamente quantos bytes ela deve mover, caso contrário, ela poderia sobrescrever dados vizinhos ou ler lixo da memória.

O "underline" é substituído por um **sufixo** que define o tamanho da operação.

| Sufixo | Nome | Tamanho | Exemplo de Registrador |
|---|---|---|---|
| b | **b**yte | 8 bits (1 byte) | al, ah, bl |
| w | **w**ord | 16 bits (2 bytes) | ax, bx, cx |
| l | **l**ong | 32 bits (4 bytes) | eax, ebx, ecx |
| q | **q**uadword | 64 bits (8 bytes) | rax, rbx, rcx |

## Exemplos Práticos

1. **movb $4, %al**
   - Move o valor imediato 4 para o registrador AL (8 bits).
   - **Por que usar 'b'?** Porque o destino (AL) só tem espaço para 1 byte.

2. **movw %ax, %bx**
   - Copia 16 bits do registrador AX para o BX.

3. **movl $0x12345678, %eax**
   - Move um valor de 32 bits para o EAX. Lembre-se: em x86-64, mover para um registrador l (32 bits) zera automaticamente a parte superior do registrador de 64 bits correspondente.

4. **movq %rax, (%rbx)**
   - Move 64 bits (8 bytes) do registrador RAX para o endereço de memória apontado por RBX.